```
import pandas as pd
import numpy as np

df = pd.read_csv("train.csv")

print("Dataset loaded successfully!")
print("Shape:", df.shape)
display(df.head())
```

```
Dataset loaded successfully!
Shape: (891, 12)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

```
# STEP 2: DATA QUALITY REPORT

print("DATA QUALITY REPORT")
print("=" * 40)

# 1. Dataset size
print("Rows:", df.shape[0])
```

```
print("Columns:", df.shape[1])

# 2. Missing values in each column
print("\nMissing Values:")
display(df.isnull().sum().to_frame("Missing Count"))

# 3. Duplicate rows
print("\nDuplicate Rows:", df.duplicated().sum())

# 4. Data types
print("\nColumn Data Types:")
display(df.dtypes.astype(str).to_frame("Data Type"))

# 5. Numeric value summary
print("\nNumeric Summary:")
display(df.describe().T)

# 6. Unique values in each column
print("\nUnique Values:")
display(df.nunique().to_frame("Unique Count"))
```



```
DATA QUALITY REPORT
==================================================
Rows: 891
Columns: 12

Missing Values:
```

	Missing Count
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

```
Duplicate Rows: 0

Column Data Types:
```

	Data Type
PassengerId	int64
Survived	int64
Pclass	int64

Name	object
Sex	object
Age	float64
SibSp	int64
Parch	int64
Ticket	object
Fare	float64
Cabin	object
Embarked	object

Numeric Summary:

	count	mean	std	min	25%	50%	75%	max
PassengerId	891.0	446.000000	257.353842	1.00	223.5000	446.0000	668.5	891.0000
Survived	891.0	0.383838	0.486592	0.00	0.0000	0.0000	1.0	1.0000
Pclass	891.0	2.308642	0.836071	1.00	2.0000	3.0000	3.0	3.0000
Age	714.0	29.699118	14.526497	0.42	20.1250	28.0000	38.0	80.0000
SibSp	891.0	0.523008	1.102743	0.00	0.0000	0.0000	1.0	8.0000
Parch	891.0	0.381594	0.806057	0.00	0.0000	0.0000	0.0	6.0000
Fare	891.0	32.204208	49.693429	0.00	7.9104	14.4542	31.0	512.3292

Unique Values:

	Unique Count
PassengerId	891
Survived	2
Pclass	3

```
# STEP 3: MISSING VALUE HANDLING

# Check missing values before cleaning
print("Missing values BEFORE cleaning:")
print(df.isnull().sum())

# PassengerId is an identifier, so don't impute it.
# Age: fill missing values with median.
# Embarked: fill missing values with mode.
# Cabin: too many missing values; retain it for now
# and document the decision.

df_clean = df.copy()

df_clean["Age"] = df_clean["Age"].fillna(
    df_clean["Age"].median()
)

df_clean["Embarked"] = df_clean["Embarked"].fillna(
    df_clean["Embarked"].mode()[0]
)

# Cabin has many missing values. Drop this column only
# after reviewing its missingness and project needs.
df_clean = df_clean.drop(columns=["Cabin"])

print("\nMissing values AFTER this step:")
print(df_clean.isnull().sum())
```

```
Missing values BEFORE cleaning:
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
```

```
Age           177
SibSp           0
Parch           0
Ticket          0
Fare            0
Cabin         687
Embarked        2
dtype: int64

Missing values AFTER this step:
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Embarked       0
dtype: int64
```

Missing Data Handling Decisions

- **Age:** Missing values were replaced with the median because age is numeric and the median is less affected by extreme values.
- **Embarked:** Missing values were replaced with the mode because it is a categorical column.
- **Cabin:** The column was removed because it contains many missing values. This decision may lose useful information.
- **PassengerId:** Kept unchanged because it is an identifier and should not be imputed.
- Other columns: Missing values will be reviewed before deciding whether to impute or remove rows.

```
# STEP 4: DUPLICATE REMOVAL
```

```
# Count duplicate rows before removal
duplicates_before = df_clean.duplicated().sum()

# Remove exact duplicate rows
df_clean = df_clean.drop_duplicates().copy()

# Count duplicate rows after removal
duplicates_after = df_clean.duplicated().sum()

print("Duplicate rows before cleaning:", duplicates_before)
print("Duplicate rows removed:", duplicates_before - duplicates_after)
print("Duplicate rows after cleaning:", duplicates_after)
print("Rows remaining:", len(df_clean))
```

```
Duplicate rows before cleaning: 0
Duplicate rows removed: 0
Duplicate rows after cleaning: 0
Rows remaining: 891
```

Duplicate Removal

Exact duplicate rows were identified using pandas `duplicated()` and removed using `drop_duplicates()`.

The number of duplicate rows removed and the remaining row count were recorded to document the cleaning process.

Only exact duplicate rows were removed. Similar-looking records were not removed without further investigation.

```
# STEP 5: STANDARDISATION AND DATA TYPE CORRECTION

# Work on a copy
df_clean = df_clean.copy()

# Standardise text formatting
df_clean["Sex"] = (
    df_clean["Sex"]
```

```
    df_clean["Sex"]
    .astype("string")
    .str.strip()
    .str.lower()
    .map({"male": "Male", "female": "Female"})
)

df_clean["Embarked"] = (
    df_clean["Embarked"]
    .astype("string")
    .str.strip()
    .str.upper()
)

# Correct data types
df_clean["PassengerId"] = df_clean["PassengerId"].astype("string")
df_clean["Age"] = pd.to_numeric(df_clean["Age"], errors="coerce")
df_clean["Fare"] = pd.to_numeric(df_clean["Fare"], errors="coerce")
df_clean["SibSp"] = pd.to_numeric(df_clean["SibSp"], errors="coerce").asty
df_clean["Parch"] = pd.to_numeric(df_clean["Parch"], errors="coerce").asty
df_clean["Pclass"] = pd.to_numeric(df_clean["Pclass"], errors="coerce").as
df_clean["Survived"] = pd.to_numeric(df_clean["Survived"], errors="coerce"

print("Standardised categories:")
print("Sex:", df_clean["Sex"].unique())
print("Embarked:", df_clean["Embarked"].unique())

print("\nUpdated data types:")
display(df_clean.dtypes.astype(str).to_frame("Data Type"))

print("\nMissing values after conversion:")
display(df_clean.isnull().sum().to_frame("Missing Count"))
```



```
Standardised categories:
Sex: ['Male' 'Female']
Embarked: <StringArray>
['S', 'C', 'Q']
Length: 3, dtype: string

Updated data types:
```

	Data Type
PassengerId	string
Survived	Int64
Pclass	Int64
Name	object
Sex	object
Age	float64
SibSp	Int64
Parch	Int64
Ticket	object

Standardisation and Data Type Correction

- Standardised the Sex column to use "Male" and "Female".
- Removed extra spaces and standardised Embarked category codes.
- Converted PassengerId to string because it is an identifier.
- Converted numeric columns to appropriate integer or decimal types.
- Checked for missing values introduced by type conversion.
- No unsupported category values were intentionally replaced.

```
# STEP 6: OUTLIER DETECTION USING IQR

numeric_cols = ["Age", "Fare", "SibSp", "Parch"]

outlier_results = []

for col in numeric_cols:
    values = df_clean[col].dropna()

    Q1 = values.quantile(0.25)
    Q3 = values.quantile(0.75)
    IQR = Q3 - Q1
```

```
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outlier_count = (
        (values < lower_bound) | (values > upper_bound)
    ).sum()

    outlier_results.append({
        "Column": col,
        "Q1": round(Q1, 2),
        "Q3": round(Q3, 2),
        "IQR": round(IQR, 2),
        "Lower Bound": round(lower_bound, 2),
        "Upper Bound": round(upper_bound, 2),
        "Outlier Count": int(outlier_count),
        "Decision": "Retain and document"
    })

outlier_report = pd.DataFrame(outlier_results)

display(outlier_report)
```

	Column	Q1	Q3	IQR	Lower Bound	Upper Bound	Outlier Count	Decision
0	Age	22.00	35.0	13.00	2.50	54.50	66	Retain and document
1	Fare	7.91	31.0	23.09	-26.72	65.63	116	Retain and document
2	SibSp	0.00	1.0	1.00	-1.50	2.50	46	Retain and document
3	Parch	0.00	0.0	0.00	0.00	0.00	213	Retain and document

Next steps: Generate code with `outlier_report` New interactive sheet

Outlier Detection and Treatment

The Interquartile Range (IQR) method was used to identify potential outliers in Age, Fare, SibSp, and Parch.

Values below Q1 - 1.5 × IQR or above Q3 + 1.5 × IQR were flagged as potential outliers.

Decision: Potential outliers are retained rather than automatically removed or capped. Extreme values may represent genuine passenger characteristics or ticket prices. The outlier counts and boundaries are documented for review.

```
# STEP 7: BEFORE VS AFTER CLEANING SUMMARY

# Original dataset: df
# Cleaned dataset: df_clean

summary = pd.DataFrame({
    "Metric": [
        "Row Count",
        "Column Count",
        "Total Missing Values",
        "Duplicate Rows"
    ],
    "Before Cleaning": [
        len(df),
        len(df.columns),
        int(df.isnull().sum().sum()),
        int(df.duplicated().sum())
    ],
    "After Cleaning": [
        len(df_clean),
        len(df_clean.columns),
        int(df_clean.isnull().sum().sum()),
        int(df_clean.duplicated().sum())
```

```
    ]
})

display(summary)

print("\nData Types After Cleaning:")
display(df_clean.dtypes.astype(str).to_frame("Data Type"))

print("\nMissing Values Remaining:")
display(df_clean.isnull().sum().to_frame("Missing Count"))
```